P16 — Wildlife Detection in Camera-Trap Images: Final Technical Summary

Computer Vision 2026, team project · Code: github.com/Seren1971/CV-Project

1. Problem and question

Camera traps produce thousands of frames per deployment; most are empty (triggered by vegetation) or hard (night IR, animals partly outside the frame, small distant animals). We evaluate how well a modern detector finds and classifies animals on **camera locations it never saw in training**, and ask a controlled question: **does augmentation targeted at the observed failure conditions beat standard augmentation at an equal training budget?** Baseline A (standard augmentation) is compared with two targeted variants (B and its fixed version B′) and an untrained COCO reference.

2. Dataset

CCT20 (Caltech Camera Traps benchmark subset, Beery et al., ECCV 2018; LILA BC, CDLA-permissive): motion-triggered images with COCO-style boxes. We keep the 11 animal classes with ≥ 100 training images (bird, bobcat, cat, coyote, dog, opossum, rabbit, raccoon, rodent, skunk, squirrel). Splits reuse the official location-based split to prevent leakage: train 11,382 images / 11,891 boxes (10 locations), val 2,148 (same locations, unseen sequences — used only for model and threshold selection), test 5,842 (**9 new locations**), test_cis 6,076 (seen locations, measures the new-location gap). Our sequence-leakage check found that the official split cuts 224 bursts between train and cis_val; the affected 270 val frames were removed. Empty frames are kept in val/test to measure vegetation false alarms directly. Each test image is auto-tagged (night IR, dark, small < 1 % of image area, truncated at the border) for per-condition analysis.

3. Method

Detector: **Faster R-CNN ResNet-50 FPN** (torchvision), COCO_V1 weights, long side 1024 px. Training: SGD 0.01 (momentum 0.9, wd 1e-4), warm-up + cosine decay, batch 4, seed 0; epoch selection by val mAP@0.5; confidence threshold = best F1 on val, then fixed for test (test is never tuned).

	Reference	A (baseline)	B	B′
Trained on CCT20	no	yes	yes	yes
Augmentation	–	flip, colour jitter, scale 640–896	A + border crop, partial erase, darken, grayscale (p = 0.3), scale 480–960	B without grayscale; darken only on colour frames; border crop keeps clipped boxes (≥ 15 % visible)
Epochs	–	6 and 12	6	12

Variant B was designed from A's failure profile (recall 0.39 on truncated and 0.43 on small animals). Stage 4 separates the training-budget effect from the augmentation effect (6 → 12 epochs) and tests B′, which fixes three diagnosed problems of B: darkening already-dark IR frames, removing colour cues needed to tell species apart, and leaving cropped animal parts in the image as unlabeled background.

4. Results

Test = new locations, class-aware metrics unless noted; thresholds from val.

Model	mAP@0.5	mAP@[.5:.95]	Precision	Recall	F1	FP / empty
COCO reference (class-agnostic)	0.377	0.183	0.706	0.356	0.473	0.023
A, 6 epochs	0.603	0.367	0.701	0.623	0.660	0.239
B, 6 epochs	0.560	0.329	0.648	0.580	0.612	0.244
A, 12 epochs	0.607	0.367	0.821	0.563	0.668	0.080
B′, 12 epochs (final)	**0.609**	**0.368**	**0.821**	**0.569**	**0.672**	**0.115**

Class-agnostic ("is there an animal"): B′ 12 ep **0.837** vs A 0.820 mAP@0.5 (F1 0.808 vs 0.781). On test_cis (seen locations) both 12-epoch models are equal (0.86 class-aware, 0.91 agnostic): the new-location gap (0.61 vs 0.86) is mostly species confusion, not missed animals. Per condition (recall, B′ vs A at 12 ep): **small animals 0.416 vs 0.364 (+5.2 pp, 95 % bootstrap CI [+3.3, +7.3] excludes 0; the gap to the full set shrinks 0.25 → 0.19)**, truncated 0.350 vs 0.333 (n.s.), night 0.654 vs 0.647 (n.s.). B′ costs +0.035 false alarms per empty frame (n.s.), concentrated in day frames.

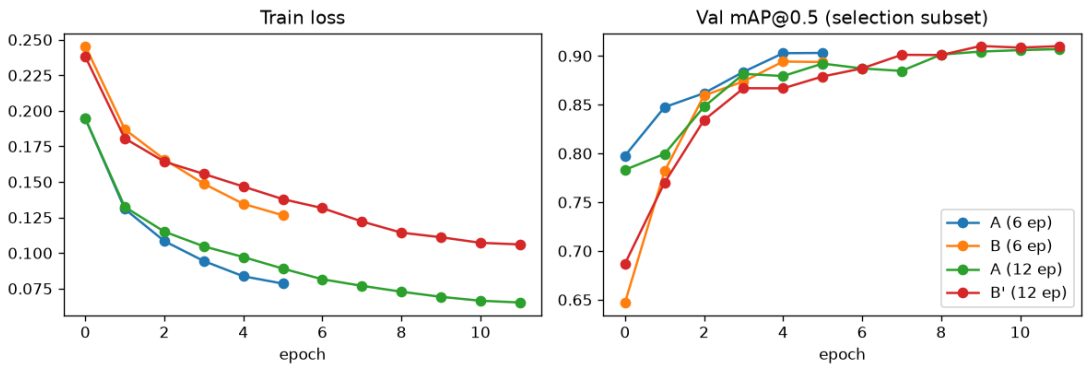


Figure 1. Training loss and val mAP@0.5: B′ (red) starts slower than A but reaches the best val mAP (0.910) by epoch 9.

5. Interpretation

At 6 epochs targeted augmentation **hurt** (–4.3 mAP); at 12 epochs the fixed variant B′ **matches the baseline overall and beats it where it was designed to**: small animals and class-agnostic detection. The budget was the main hidden factor — B's val mAP lagged A's until epoch 6 and only converged later. Longer training also fixes calibration: false alarms on empty frames drop 3× (0.24 → 0.08 per image) and the share of correct detections rises from 70 % to 82 %. Night turned out not to be a bottleneck after fine-tuning (night ≈ day recall; the raw difference comes from the species mix), which is why B's night-oriented transforms did not pay off.

6. Failure analysis

Error taxonomy on test (thresholded detections, final model B′ 12 ep): **82 % correct, 8 % class confusion, 10 % background false positives; 37 % of ground-truth animals missed** (at the val-chosen threshold 0.9). Representative cases: (1) **class confusion** — a night close-up of a cat (rear view, no face) is predicted as coyote with score 1.0; (2) **missed truncated animal** — an animal filling the whole frame at close range is missed entirely; border-crop augmentation only partly imitates this; (3) **background false positive** — sun glare on water in an empty frame is detected as an animal (0.95); moving water and vegetation remain the main false-alarm source; (4) **small animals** — rodent AP@0.5 ≈ 0.04–0.09: boxes under 1 % of the image lose detail on 1024-px inputs, and B′ recovers only part of this gap.

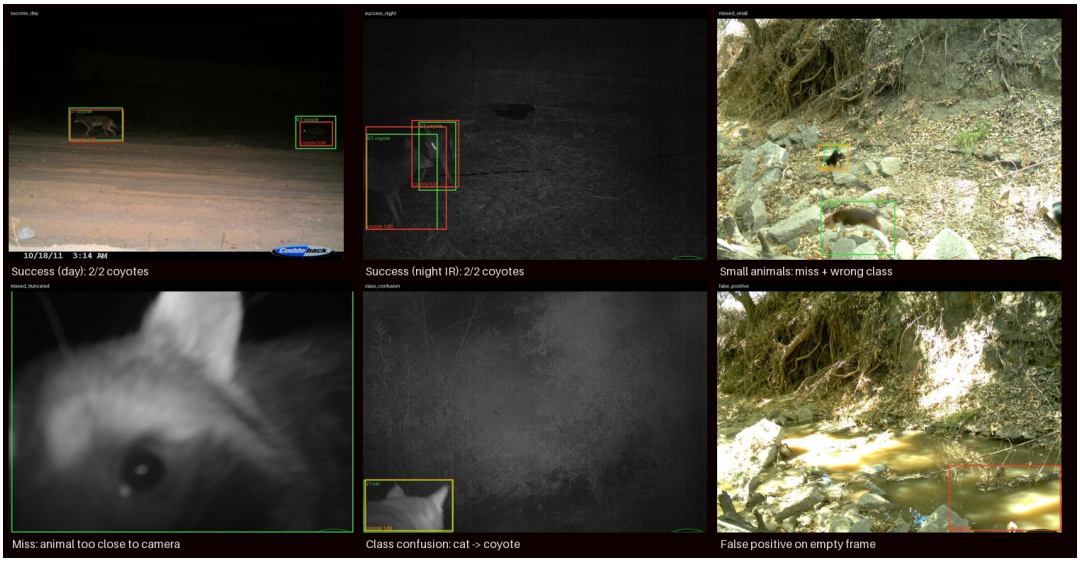


Figure 2. Successes and representative failures of the final model (green = ground truth, red/orange = predictions).

7. Reproducibility and limitations

Repo: github.com/Seren1971/CV-Project — README with exact commands, requirements.txt, one-click run_baseline.bat / run_stage4.bat (Windows) or python scripts/run_all.py; committed splits in data/splits/; metrics and figures under results/. Environment: Python 3.13.5, PyTorch 2.11.0+cu128, RTX 5070; training 115–118 min per 12-epoch run, inference ≈ 0.1 s/image (results/environment.txt). Limitations: one seed per configuration (run-to-run spread unmeasured, differences < 1 mAP point are noise); resized "_sm" images hurt small-animal detail; augmentation effects are attributed to the transform package, not single transforms; CCT20 covers one region and 11 classes.

8. Team contributions

Sergey Peshkov — dataset selection and preparation, leakage-free splits, condition tagging, training/evaluation pipeline, experiment runs (A, B, B′), failure-case rendering, demo, repository. **Arina Nikolaeva** — model and implementation specification: Faster R-CNN R50-FPN choice, pretrained weights, software stack and GPU pipeline. **Anton Kropotov** — experimental design: hypotheses, run configuration, metrics protocol, targeted-augmentation idea, error-analysis protocol. **avlaptev** — project scope and documentation: research question, boundaries, plan assembly, consistency checks. All members reviewed the integrated plan and prepared to explain model choice, metrics, results and failure cases individually.